LLM Inference Performance Lab

Project type: ML Infrastructure / AI Systems Engineering
Estimated development time: 4–6 weeks
Primary language: Python
Core technologies: vLLM, PyTorch, FastAPI, Docker, Prometheus, Grafana, NVIDIA GPU monitoring
Supporting technologies: PostgreSQL or Parquet, pandas, NumPy, matplotlib, pytest, GitHub Actions
Target roles: ML Infrastructure Engineer, AI Infrastructure Engineer, Backend SWE, Inference Engineer

---

1. Executive Summary

1.1 Project Overview

The **LLM Inference Performance Lab** is a benchmarking, profiling, and optimization platform for evaluating the performance of self-hosted large language models under realistic serving workloads.

The system will deploy open-weight language models using vLLM, generate reproducible inference workloads, collect model-serving and GPU metrics, compare serving configurations, and produce automated performance reports.

The platform will focus on four critical aspects of LLM inference:

1. **Latency:** How quickly a model begins generating and completes a response.
2. **Throughput:** How many requests and output tokens the system processes per second.
3. **GPU efficiency:** How effectively GPU compute and memory resources are utilized.
4. **Performance tradeoffs:** How batching, concurrency, context length, quantization, and scheduling influence system behavior.

Rather than implementing a new transformer inference engine, this project will investigate and optimize an existing production-oriented inference runtime.

The final deliverable will be a reproducible benchmarking environment with automated workload generation, telemetry collection, experiment comparisons, performance regression checks, and a technical report documenting measured bottlenecks and improvements.

1.2 Problem Statement

Serving an LLM efficiently is fundamentally different from invoking a hosted model API.

A self-hosted inference system must manage GPU memory, attention caches, concurrent requests, variable sequence lengths, and competing latency and throughput objectives.

A serving configuration that performs well for a single request may degrade significantly when handling concurrent workloads.

For example:

- Larger batches can improve aggregate throughput but increase queuing delays.
- Long prompts can increase prefill computation and KV-cache memory requirements.
- More concurrent requests can improve GPU utilization until memory or scheduling limits are reached.
- Quantization can reduce memory consumption but may change accuracy and performance.
- A configuration optimized for throughput may violate interactive latency requirements.

Without controlled benchmarking and profiling, engineers cannot reliably identify which configuration is appropriate for a given workload.

This project addresses that problem through repeatable experiments and evidence-based performance analysis.

1.3 Primary Objective

Build a reproducible LLM inference benchmarking platform capable of measuring, comparing, and explaining serving performance across different workload patterns and runtime configurations.

The system should answer questions such as:

- How does concurrency affect request throughput and p95 latency?
- At what point does increasing concurrency stop improving useful throughput?
- How does prompt length affect time to first token?
- How do output lengths affect decode performance?
- How does vLLM's maximum batched-token setting influence throughput and tail latency?
- How much GPU memory is consumed as active sequence count increases?
- Does quantization improve useful throughput under a fixed GPU memory budget?
- Which serving configuration meets a specified latency service-level objective at the highest sustainable request rate?

2. Scope and Non-Goals

2.1 In Scope

The project will implement:

- Self-hosted inference using vLLM.
- An OpenAI-compatible inference endpoint.
- Reproducible synthetic workload generation.
- Request concurrency and arrival-rate control.
- Streaming response timing.
- Request-level latency and token metrics.
- GPU utilization and memory telemetry.
- Automated benchmark execution.
- Configuration comparison and performance reporting.
- Latency/throughput regression checks.
- Failure and saturation analysis.
- Docker-based local or cloud deployment.
- Reproducible experiment manifests and results.

2.2 Out of Scope

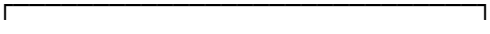
The initial project will not attempt to:

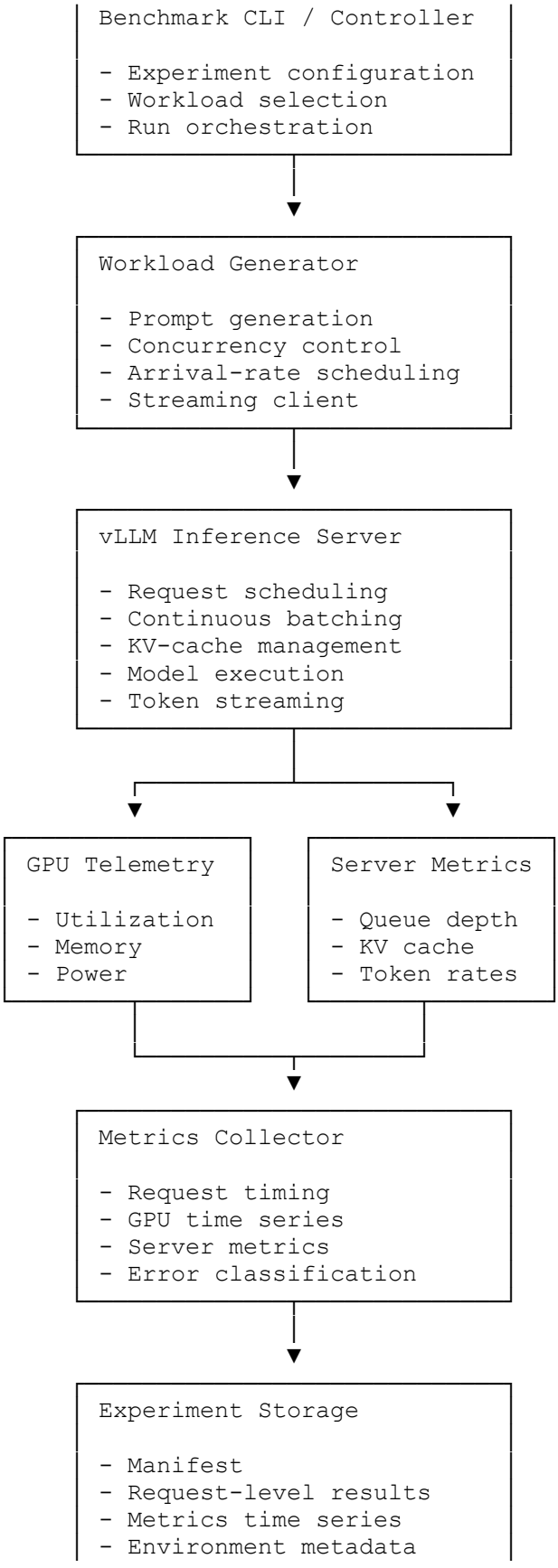
- Train or fine-tune an LLM.
- Implement transformer attention kernels from scratch.
- Build a replacement for vLLM.
- Implement distributed tensor or pipeline parallelism.
- Build a Kubernetes autoscaling platform.
- Claim production-scale performance from a single-GPU benchmark.
- Guarantee exact reproducibility of GPU execution timing.
- Optimize model answer quality as its primary objective.

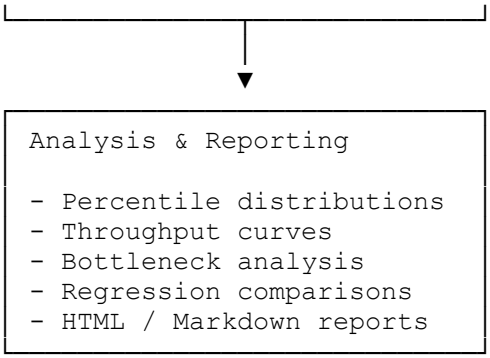
A small correctness-validation component may be added to ensure that optimizations do not introduce unacceptable output degradation.

3. Proposed System Architecture

3.1 High-Level Architecture




Benchmark CLI / Controller
- Experiment configuration
- Workload selection
- Run orchestration
Workload Generator
- Prompt generation
- Concurrency control
- Arrival-rate scheduling
- Streaming client
vLLM Inference Server
- Request scheduling
- Continuous batching
- KV-cache management
- Model execution
- Token streaming
GPU Telemetry
- Utilization
- Memory
- Power
Server Metrics
- Queue depth
- KV cache
- Token rates
Metrics Collector
- Request timing
- GPU time series
- Server metrics
- Error classification
Experiment Storage
- Manifest
- Request-level results
- Metrics time series
- Environment metadata



3.2 Component Responsibilities

Benchmark Controller

The controller loads experiment definitions, validates configuration, starts workloads, coordinates telemetry collection, and writes experiment metadata.

It should support:

- Repeated runs.
- Warm-up periods.
- Multiple workload configurations.
- Fixed random seeds.
- Experiment timeouts.
- Clean shutdown and partial-result preservation.
- Hardware and software version recording.

Workload Generator

The workload generator creates requests with controlled characteristics.

It must support both:

Closed-loop workloads: A fixed number of concurrent clients continuously issue requests.

Open-loop workloads: Requests arrive according to a configured schedule or rate, independently of previous request completion.

This distinction is important because a fixed-concurrency benchmark can hide queuing behavior under overload.

Inference Server

vLLM will serve the selected model through an OpenAI-compatible API.

The initial implementation will use a single model on a single GPU.

The platform will treat vLLM as an external serving engine rather than modifying its internal scheduling implementation.

Metrics Collector

The metrics collector combines:

- Client-side request measurements.
- Server-side Prometheus metrics.
- NVIDIA GPU telemetry.

Measurements will use consistent timestamps and preserve raw observations for later analysis.

Reporting Engine

The reporting engine calculates aggregate metrics, generates charts, compares configurations, and produces reproducible benchmark summaries.

4. Technology Stack

Layer	Technology	Purpose
Language	Python 3.11+	Benchmarking and orchestration
Inference	vLLM	LLM serving and continuous batching
Model framework	PyTorch	Model runtime and GPU integration
API protocol	OpenAI-compatible HTTP	Standardized inference requests
HTTP client	httpx	Async requests and streaming
Workload control	asyncio	Concurrency and arrival scheduling
GPU telemetry	NVIDIA NVML / DCGM	GPU utilization, memory and power
Metrics	Prometheus	Server metrics collection
Dashboard	Grafana	Performance visualization
Data analysis	pandas, NumPy	Benchmark aggregation
Charts	matplotlib	Performance reports
Storage	Parquet and JSON	Raw observations and metadata
Containers	Docker	Reproducible deployment
Testing	pytest	Unit and integration testing
CI	GitHub Actions	CPU-only validation and report checks

A database is optional. Parquet and JSON are sufficient for the initial version and avoid unnecessary operational complexity.

5. Model and Hardware Selection

5.1 Initial Model

Use a small open-weight instruction-tuned model, such as a compatible 1B–3B parameter model.

Candidate model families include:

- Qwen small instruction models.
- Llama small instruction models, subject to access and license requirements.
- Other vLLM-supported models appropriate for the available GPU.

The exact model revision must be pinned before benchmarking.

Avoid using a large model until the measurement pipeline is validated.

5.2 Hardware Requirements

A CUDA-capable NVIDIA GPU is the preferred target.

Environment	Suitability
CPU-only laptop	Workload generator, analysis, tests
Apple Silicon Mac	Client development; alternative inference backends possible
NVIDIA GPU with approximately 16–24 GB VRAM	Good initial single-GPU target for appropriately sized models
Cloud NVIDIA GPU	Suitable for controlled performance studies
Multi-GPU server	Optional future extension

Memory requirements depend on model size, precision, context length, KV-cache allocation, and runtime overhead. The project must verify actual fit instead of relying on nominal VRAM alone.

If no NVIDIA GPU is available locally, develop the client and reporting components on your existing machine, then run GPU experiments on a rented instance.

5.3 Reproducibility Requirements

Every experiment must record:

- GPU model and total VRAM.
- GPU driver version.
- CUDA and PyTorch versions.
- vLLM version.
- Model identifier and revision.
- Quantization and dtype.
- Maximum model context length.
- Relevant serving flags.
- Container image digest.
- Benchmark workload seed.
- Workload generation configuration.
- Experiment timestamp.
- Source Git commit.

GPU clock behavior, thermal conditions, and shared cloud hardware may still introduce variability.

6. Workload Generator Design

6.1 Workload Parameters

Each benchmark should support the following configuration:

Parameter	Example values
Concurrency	1, 4, 8, 16, 32
Arrival rate	1, 2, 4, 8 requests/sec
Input length	128, 512, 2,048 tokens
Requested output length	64, 128, 256 tokens
Request count	100–1,000 per configuration
Temperature	0
Streaming	Enabled
Timeout	Configurable
Random seed	Fixed
Model	Pinned model revision

These values are starting points, not mandatory production settings. Reduce them if the hardware cannot sustain the workload.

6.2 Prompt Generation

The workload generator will produce token-length-controlled prompts.

Requirements:

1. Use the model's actual tokenizer.
2. Construct prompts near the requested token length.
3. Record actual token counts.
4. Preserve the generated workload.
5. Reuse identical prompts when comparing configurations.
6. Avoid placing large random strings in prompts without checking tokenization behavior.

The system should distinguish requested output limits from actual generated output tokens.

6.3 Workload Modes

Mode A — Fixed Concurrency

Maintain a specified number of active clients.

Purpose:

- Measure throughput scaling.
- Evaluate batching benefits.
- Identify resource saturation.

Mode B — Fixed Arrival Rate

Schedule requests at a controlled rate regardless of response completion.

Purpose:

- Measure queuing delays.
- Observe latency degradation.
- Identify sustainable throughput.
- Evaluate overload behavior.

The generator should record both scheduled arrival time and actual dispatch time so client scheduling delays are visible.

Mode C — Mixed Workloads

Combine short and long prompts and outputs.

Example workload:

- 60% short requests.
- 30% medium requests.
- 10% long-context requests.

Purpose:

- Observe head-of-line effects.
- Examine fairness between short and long requests.
- Measure tail latency under heterogeneous demand.

7. Performance Metrics

7.1 Time to First Token (TTFT)

TTFT measures the time from sending a request to receiving the first generated output token.

It includes network overhead, server queuing, prefill, and initial generation.

For streaming APIs, the client must distinguish content-bearing token events from headers, role-only events, and other non-token events.

Report:

- p50 TTFT.
- p95 TTFT.
- p99 TTFT when sample size supports it.

7.2 Time per Output Token (TPOT)

For a response with at least two output tokens:

TPOT = (time of last output token − time of first output token) / (number of output tokens − 1)

Report TPOT distributions across requests.

Token chunks are not necessarily one token each. Prefer token-aware server metrics or verified token counts rather than assuming each streaming event contains exactly one token.

7.3 End-to-End Latency

Measure the time between scheduled or actual request dispatch and complete response receipt.

Report both:

- Client-observed service latency.
- Schedule-to-completion latency, including client dispatch delay.

This distinction becomes important under high load.

7.4 Throughput

Measure:

Requests per second

Completed requests divided by the relevant measurement interval.

Output tokens per second

Total generated output tokens divided by the measurement interval.

Total processed tokens per second

Input and output tokens processed per unit time, labeled separately where possible.

Do not equate configured concurrency with achieved throughput.

7.5 GPU Utilization

Collect:

- GPU compute utilization.
- Used and total GPU memory.
- GPU power consumption when available.
- GPU temperature when useful.
- GPU memory bandwidth utilization when supported.

Sampling intervals and measurement limitations must be recorded.

7.6 KV-Cache and Scheduler Metrics

Where exposed by the installed vLLM version, collect:

- KV-cache utilization.
- Waiting requests.

- Running requests.
- Scheduling or preemption events.
- Prompt token processing.
- Generation token processing.

Metric names and availability are version-dependent and must be verified against the actual runtime.

7.7 Reliability Metrics

Collect:

- Successful requests.
- Failed requests.
- Timeouts.
- Rate-limit responses.
- Out-of-memory failures.
- Canceled requests.
- Incomplete streams.
- Invalid responses.

Failed and timed-out requests must not disappear from the denominator of reported results.

8. Experiment Design

The core project will contain five experiments.

Experiment 1 — Concurrency Scaling

Research Question

How does concurrent request count affect throughput and latency?

Configuration

Hold constant:

- Model and revision.
- GPU.
- Prompt distribution.
- Output limits.
- vLLM configuration.

Vary:

- Concurrency: 1, 4, 8, 16, 32.

Measurements

- Requests/sec.
- Output tokens/sec.
- p50/p95 TTFT.
- p50/p95 end-to-end latency.
- GPU utilization.
- GPU memory.
- Error rate.

Expected Analysis

Identify where throughput begins to plateau while latency continues to increase.

Do not assume the highest concurrency setting is optimal.

Deliverable

Concurrency-versus-throughput and concurrency-versus-latency curves.

Experiment 2 — Prompt Length and Prefill Cost

Research Question

How does input context length affect inference latency?

Configuration

Use input lengths approximately:

- 128 tokens.
- 512 tokens.
- 2,048 tokens.
- 4,096 tokens if supported by hardware.

Keep output limits and concurrency constant.

Measurements

- TTFT.
- Prefill-related server metrics.
- Total latency.
- GPU memory.
- Output throughput.

Expected Analysis

Determine whether longer prompts increase TTFT and whether the effect changes under concurrency.

Deliverable

Prompt-length-versus-TTFT report.

Experiment 3 — Batching and Scheduling

Research Question

How do vLLM batching parameters affect throughput and tail latency?

Configuration

Compare selected values of:

- `max_num_batched_tokens`.
- `max_num_seqs`.

Use settings supported by the installed vLLM release.

Measurements

- Throughput.
- p95 TTFT.
- p95 end-to-end latency.
- Waiting and running requests.
- KV-cache utilization.
- GPU utilization.

Expected Analysis

Explain the tradeoff between throughput-oriented and latency-oriented configurations.

Deliverable

A configuration comparison identifying the best measured settings for the selected workload and hardware.

Experiment 4 — KV-Cache and Memory Pressure

Research Question

How does increasing active context volume affect GPU memory usage and inference performance?

Configuration

Increase combinations of:

- Active sequence count.
- Prompt length.
- Maximum model context.
- GPU memory utilization target.

Measurements

- GPU memory usage.
- KV-cache utilization.
- Running/waiting requests.
- Preemption events where available.
- Throughput.
- Latency.
- OOM or admission failures.

Expected Analysis

Identify memory-related saturation and explain how it affects request scheduling.

Deliverable

Memory-pressure and performance analysis.

Experiment 5 — Quantization Tradeoffs

Research Question

Does lower-precision model execution improve serving efficiency on the available GPU?

Configuration

Compare compatible model representations, such as:

- BF16/FP16 baseline.
- A supported quantized representation.

Use compatible model weights, kernels, and hardware.

Measurements

- GPU memory usage.
- Throughput.
- TTFT.
- TPOT.
- Error rate.
- Optional answer-quality checks.

Expected Analysis

Evaluate whether memory savings translate into higher concurrency or throughput, and whether output quality changes.

Quantization must not be assumed to improve latency automatically.

Deliverable

Precision-versus-performance comparison.

This experiment is optional if compatible quantized models or adequate GPU memory are unavailable.

9. Automated Benchmark Execution

9.1 Experiment Configuration

Define experiments using YAML.

Example:

```
experiment:
  name: concurrency-scaling
  repetitions: 3
  warmup_requests: 20
  random_seed: 42

model:
  id: pinned-model-id
  revision: pinned-revision

workload:
  mode: closed_loop
  requests: 300
  input_tokens: 512
  max_output_tokens: 128
  concurrency: [1, 4, 8, 16, 32]
  streaming: true

metrics:
  collect_gpu: true
  collect_server: true
  sample_interval_ms: 500

report:
  percentiles: [50, 95, 99]
```

The controller will validate the configuration and expand it into individual benchmark runs.

9.2 Benchmark Lifecycle

Each run follows:

1. Validate the environment.
2. Record hardware and software versions.
3. Start or verify the inference server.
4. Confirm model readiness.
5. Warm up the server.
6. Start telemetry collection.
7. Execute the measured workload.
8. Stop telemetry collection.
9. Persist request-level results.
10. Calculate aggregate statistics.
11. Generate plots and summaries.
12. Cleanly release resources.

A failed run should preserve available observations and record why it was incomplete.

9.3 Repetitions

Run each important configuration at least three times where practical.

Report:

- Per-run results.
- Median across runs.
- Observed variation.
- Configuration-specific failure counts.

Avoid treating thousands of correlated requests within one run as thousands of independent hardware replications.

10. Data Storage and Result Format

10.1 Suggested Directory Structure

```
llm-inference-performance-lab/
├── README.md
├── pyproject.toml
├── Dockerfile
├── docker-compose.yml
├── configs/
│   ├── concurrency.yaml
│   ├── prompt-length.yaml
│   ├── batching.yaml
│   ├── memory-pressure.yaml
│   └── quantization.yaml
├── src/
│   └── inference_lab/
│       ├── cli.py
│       ├── config.py
│       ├── controller.py
│       ├── workload.py
│       ├── client.py
│       ├── metrics.py
│       ├── gpu_monitor.py
│       ├── analysis.py
│       └── reporting.py
├── tests/
│   ├── test_config.py
│   ├── test_workload.py
│   ├── test_streaming.py
│   ├── test_metrics.py
│   └── test_reporting.py
├── dashboards/
│   └── grafana/
├── scripts/
│   ├── start_server.sh
│   └── verify_gpu.sh
```

```
├── results/
│   └── .gitkeep
├── reports/
│   └── .gitkeep
└── docs/
    ├── architecture.md
    ├── methodology.md
    ├── experiments.md
    └── limitations.md
```

10.2 Request-Level Data

Store one record per request containing:

```
{
  "request_id": "req-001",
  "experiment_id": "exp-001",
  "scheduled_at_ms": 0,
  "dispatched_at_ms": 4,
  "completed_at_ms": 1234,
  "input_tokens": 512,
  "output_tokens": 128,
  "ttft_ms": 210,
  "tpot_ms": 7.9,
  "latency_ms": 1230,
  "status": "success",
  "error_type": null
}
```

These are illustrative values, not benchmark results.

The schema should also support missing TTFT or TPOT for failures, empty responses, and incomplete streams.

10.3 Experiment Artifacts

Each run should generate:

- `manifest.json`
- `requests.parquet`
- `gpu_metrics.parquet`
- `server_metrics.parquet`
- `summary.json`
- `report.md`

Raw data should be retained so charts can be regenerated without rerunning inference.

11. Dashboard and Visualization

11.1 Grafana Dashboard

Create a dashboard displaying:

- Active requests.
- Waiting requests.
- Request throughput.
- Output-token throughput.
- GPU utilization.
- GPU memory usage.
- KV-cache utilization.
- Request latency percentiles.
- Failure counts.

Use Prometheus as the source for server and GPU time-series metrics.

11.2 Benchmark Reports

Generate charts for:

1. Concurrency vs. throughput.
2. Concurrency vs. p95 TTFT.
3. Prompt length vs. TTFT.
4. Batch configuration vs. throughput.
5. KV-cache utilization vs. tail latency.
6. GPU memory vs. concurrent sequences.
7. Quantization vs. throughput and memory.

Every chart must identify the model, hardware, workload, and measurement units.

11.3 Comparative Reports

Support comparisons between two experiment manifests.

The report should identify:

- Throughput changes.
- Latency changes.
- Resource usage differences.
- Error-rate changes.
- Whether workloads are sufficiently comparable.
- Missing or incomplete measurements.

Do not calculate a meaningful performance improvement percentage across incompatible hardware or workload definitions without an explicit qualification.

12. Performance Regression Detection

The platform will support benchmark-based regression checks.

Example policy:

```
regression:
  baseline: results/baseline/summary.json

  rules:
    - metric: output_tokens_per_second
      minimum_ratio: 0.95

    - metric: p95_ttft_ms
      maximum_ratio: 1.10

    - metric: error_rate
      maximum: 0.01
```

A run may be marked:

- **PASS:** All required metrics meet the configured thresholds.
- **FAIL:** A valid comparison violates a threshold.
- **INCONCLUSIVE:** Required data is missing or the workloads are incompatible.

These thresholds are examples. They should be tuned to observed run-to-run variability.

GPU benchmarks should not run automatically on ordinary CPU-only GitHub Actions runners.

CI can validate the benchmarking code and compare committed benchmark artifacts; actual GPU performance checks should run on dedicated hardware.

13. Reliability and Correctness

Performance results are not meaningful if requests fail silently or output accounting is wrong.

13.1 Required Tests

Workload Generator Tests

Verify:

- Correct concurrency limits.
- Correct request count.
- Fixed-seed reproducibility.
- Open-loop scheduling.
- Cancellation behavior.
- Timeout handling.

Streaming Tests

Verify:

- First content-token timing.
- Multi-token streaming chunks.
- Empty streams.
- Interrupted streams.
- Missing usage information.
- Final response handling.

Metrics Tests

Verify:

- Percentile calculations.
- Token accounting.
- Throughput denominators.
- Failed-request inclusion.
- Handling of missing observations.

Integration Tests

Use a mock OpenAI-compatible streaming server to test the entire benchmark pipeline without requiring a GPU.

GPU Integration Tests

On real GPU hardware, verify:

- Model readiness.
- Correct inference responses.
- Valid server metrics.
- GPU telemetry collection.
- Graceful shutdown.
- Artifact generation.

13.2 Correctness Guardrails

For quantization and configuration comparisons, use a fixed sample of prompts to detect:

- Empty outputs.
- Unexpected truncation.
- Invalid structured outputs.
- Major task-specific quality regressions.

The goal is not to reproduce EvalScope's full evaluation engine.

If desired, export selected model responses to EvalScope for quality analysis instead of duplicating its functionality.

14. Development Roadmap

Week 1 — Foundation and Benchmark Client

Objective: Build a reliable benchmark generator and validate it against a mock server.

Tasks:

- Initialize repository and Python package.
- Define configuration schema.
- Implement async HTTP streaming client.
- Implement closed-loop concurrency.
- Implement open-loop arrival scheduling.
- Capture request-level timing and token usage.
- Implement JSON/Parquet artifact storage.
- Write unit and mock-server integration tests.

Deliverable: A working benchmark CLI that produces trustworthy request-level results without requiring GPU hardware.

Acceptance criteria:

- Configured request counts match actual attempts.
- Failed requests are accounted for.
- Streaming timings are validated.
- Repeated seeded workloads produce equivalent inputs.

Week 2 — Real GPU Inference and Telemetry

Objective: Execute a real model and collect hardware measurements.

Tasks:

- Provision or configure NVIDIA GPU access.
- Install a pinned vLLM release.
- Select and pin a compatible model.
- Start OpenAI-compatible serving.
- Collect vLLM Prometheus metrics.
- Collect NVIDIA GPU telemetry.
- Implement readiness checks and warm-up.
- Record environment metadata.
- Run initial single-request and concurrency tests.

Deliverable: First reproducible real-GPU benchmark.

Acceptance criteria:

- At least one model serves real inference requests.
- TTFT and latency are recorded.
- GPU memory and utilization are captured.
- Results include hardware and software metadata.

Week 3 — Controlled Performance Experiments

Objective: Measure scaling and identify bottlenecks.

Tasks:

- Implement concurrency matrix.
- Implement prompt-length matrix.
- Compare batching configurations.
- Run each important configuration repeatedly.
- Capture throughput, latency, memory, and errors.
- Analyze saturation behavior.
- Generate automated charts.

Deliverable: At least three completed real-GPU experiments with reproducible reports.

Acceptance criteria:

- Workloads are comparable.
- Raw data is preserved.
- Throughput and latency curves are generated.
- Bottleneck explanations are supported by measured data.

Week 4 — Optimization and Reporting

Objective: Demonstrate an evidence-based serving improvement.

Tasks:

- Select a baseline configuration.
- Identify the limiting resource.
- Change one serving parameter at a time.
- Measure throughput and latency changes.
- Evaluate whether latency objectives are maintained.
- Add automated comparison reports.
- Implement performance regression checks.
- Publish benchmark methodology and results.

Deliverable: A documented before/after serving configuration comparison.

Acceptance criteria:

- Baseline and candidate workloads are equivalent.
- Performance differences are measured across repeated runs.
- Failures and incomplete measurements are disclosed.
- The report explains the tradeoff, not just the percentage change.

Week 5 — Optional Advanced Experiments

Choose one:

Option A: Quantization

Compare model memory, throughput, and output quality between supported precisions.

Option B: Mixed Workload Fairness

Investigate whether long requests increase tail latency for short requests.

Option C: Memory Pressure

Analyze KV-cache pressure, scheduling behavior, and preemption.

Option D: GPU Profiling

Use supported profiling tools to examine execution time and memory behavior.

Only add an advanced experiment after the basic benchmarking system produces reliable results.

Week 6 — Portfolio and Interview Preparation

Tasks:

- Clean up documentation.
- Publish reproducible commands.
- Add architecture diagrams.
- Include measured charts and limitations.
- Record a short demonstration.
- Prepare system-design explanations.
- Write evidence-backed resume bullets.

Deliverable: Portfolio-ready repository and technical case study.

15. Minimum Viable Product

The MVP should include:

1. A reproducible async benchmarking client.
2. Closed-loop concurrency testing.
3. Open-loop arrival-rate testing.
4. Real vLLM inference on one GPU.
5. TTFT, latency, throughput, and error measurements.
6. GPU memory and utilization telemetry.
7. Three concurrency configurations.
8. Automated Markdown performance report.
9. Unit and integration tests.
10. A published real-GPU benchmark with complete hardware metadata.

Do not begin with a React dashboard, Kubernetes deployment, or multi-GPU support.

Those features would add implementation effort without proving the core performance engineering objective.

16. Success Criteria

The project is complete when it can demonstrate all of the following:

Category	Success criterion

Functional	Executes real inference requests against a self-hosted model
Measurement	Captures latency, throughput, errors and token counts
Hardware	Collects real GPU utilization and memory metrics
Experimentation	Runs repeatable workload/configuration comparisons
Optimization	Identifies and explains at least one measured performance tradeoff
Reliability	Preserves failed and incomplete requests in reports
Reproducibility	Publishes model, GPU, runtime, workload and version metadata
Testing	Validates benchmark logic without requiring a GPU
Documentation	Includes a technical case study with real measurements

There is no predetermined throughput improvement target. A valid finding that a configuration provides no benefit—or causes a regression—is still a successful experiment.

17. Engineering Challenges and Learning Outcomes

Challenge 1 — Measuring Streaming Latency Correctly

Streaming APIs may emit metadata or multiple tokens per chunk.

The project must avoid treating the first HTTP response byte as the first generated token.

Learning outcome: Precise latency measurement in asynchronous streaming systems.

Challenge 2 — Understanding GPU Saturation

Throughput can stop increasing even as request concurrency grows.

Learning outcome: Identify compute, memory, and scheduling bottlenecks using multiple correlated measurements.

Challenge 3 — Controlling Benchmark Bias

Warm-up behavior, request ordering, shared hardware and variable output lengths can distort comparisons.

Learning outcome: Design reproducible experiments and interpret noisy performance measurements.

Challenge 4 — Understanding KV-Cache Memory

Concurrent sequences compete for finite cache capacity.

Learning outcome: Understand how context length, active sequences and attention-cache allocation affect inference serving.

Challenge 5 — Balancing Latency and Throughput

A throughput-optimal configuration may provide unacceptable interactive response times.

Learning outcome: Optimize systems against workload-specific objectives rather than a single benchmark score.

18. Potential Resume Experience

LLM Inference Performance Lab
Python, vLLM, PyTorch, Docker, Prometheus, Grafana, NVIDIA GPU

Suggested bullets after implementation:

- Developed a reproducible LLM inference benchmarking platform using asynchronous workload generation and streaming telemetry to measure throughput, time to first token, tail latency and GPU utilization.
- Profiled vLLM serving performance across concurrency, prompt length and batching configurations, identifying GPU memory and request-scheduling bottlenecks through controlled experiments.
- Automated performance comparisons and regression checks using versioned experiment manifests, GPU metrics and reproducible reports.

Replace the second bullet with a specific measured result once experiments are complete. Do not claim improvements, scale, or GPU optimization before validating them.

19. Relationship to Existing Portfolio

Distributed Task Queue

Demonstrates:

- Reliable asynchronous processing.
- Worker coordination.
- Transactional persistence.
- Failure recovery.
- Concurrency control.

EvalScope

Demonstrates:

- LLM evaluation.
- Model quality measurement.
- Prompt comparison.
- Observability.
- Cost and token accounting.
- Evaluation reliability.

LLM Inference Performance Lab

Will demonstrate:

- Self-hosted model serving.
- GPU performance measurement.
- Continuous batching and scheduling analysis.
- KV-cache and memory behavior.
- Throughput and tail-latency optimization.
- Hardware-aware performance engineering.

Together, these projects cover three distinct engineering areas: **reliable workload execution, AI evaluation infrastructure, and efficient model inference**.

---

20. Final Deliverables

The finished repository should contain:

- A benchmark CLI and reusable Python package.
- Workload configuration files.
- A streaming inference client.
- Real GPU and server telemetry collectors.
- Automated performance analysis.
- Benchmark comparison and regression checks.
- Mock-server and GPU integration tests.

- Reproducible experiment manifests.
- Raw benchmark measurements.
- Performance charts.
- A technical case study.
- Architecture and methodology documentation.
- Clear instructions for reproducing the experiments.

Definition of done: A reviewer can inspect the repository, understand the workload and hardware, reproduce the methodology on compatible equipment, and verify that every reported performance claim is supported by collected measurements.

The final project should demonstrate not only that you can run an LLM, but that you can systematically measure and improve the efficiency of the infrastructure serving it.